

OBJECT ORIENTED PROGRAMMING USING C++



Study Materials

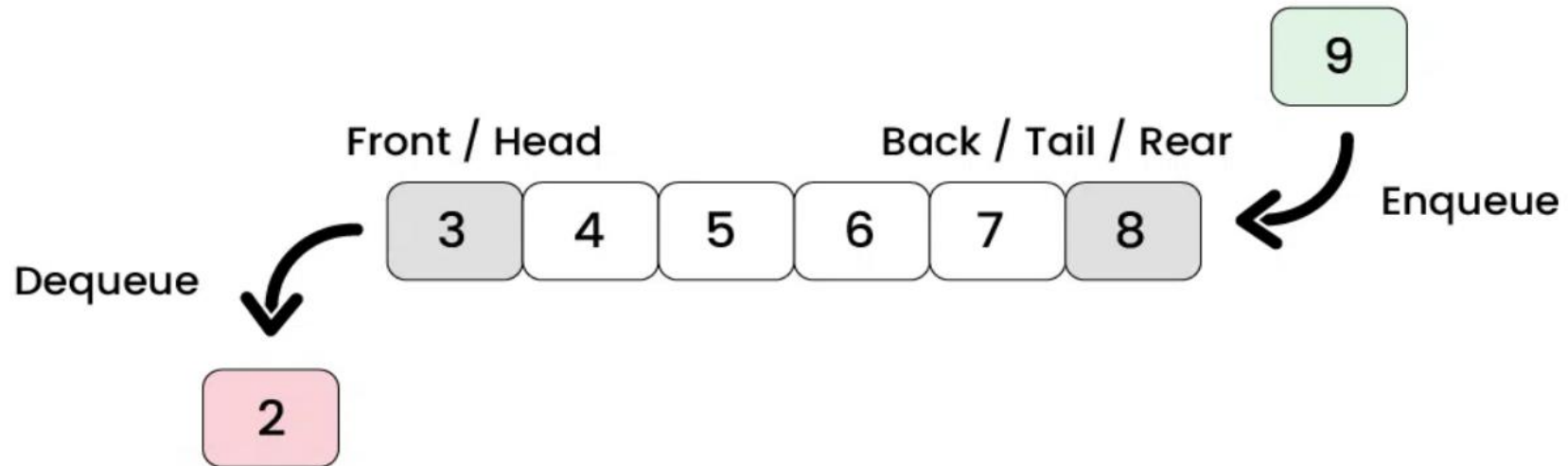
Queue:

- A queue is a linear data structure that follows the First In, First Out (FIFO) principle.
- Elements are inserted from one end called the rear and removed from the other end called the front.
- Basic Operations:
 - Enqueue: Add an element to the rear of the queue.
 - Dequeue: Remove an element from the front of the queue.
 - Front: View the element at the front without removing it.
 - isEmpty: Check if the queue is empty.

Queue Representation:

- Queue Representation:
- Front: The end where elements are removed.
- Rear: The end where elements are added.
- Visual Representation:
 - [Insert Image of a queue with elements enqueued at the rear and dequeued from the front]
- Key Points:
 - The first element added is the first one removed.
 - The queue can be implemented using arrays or linked lists.

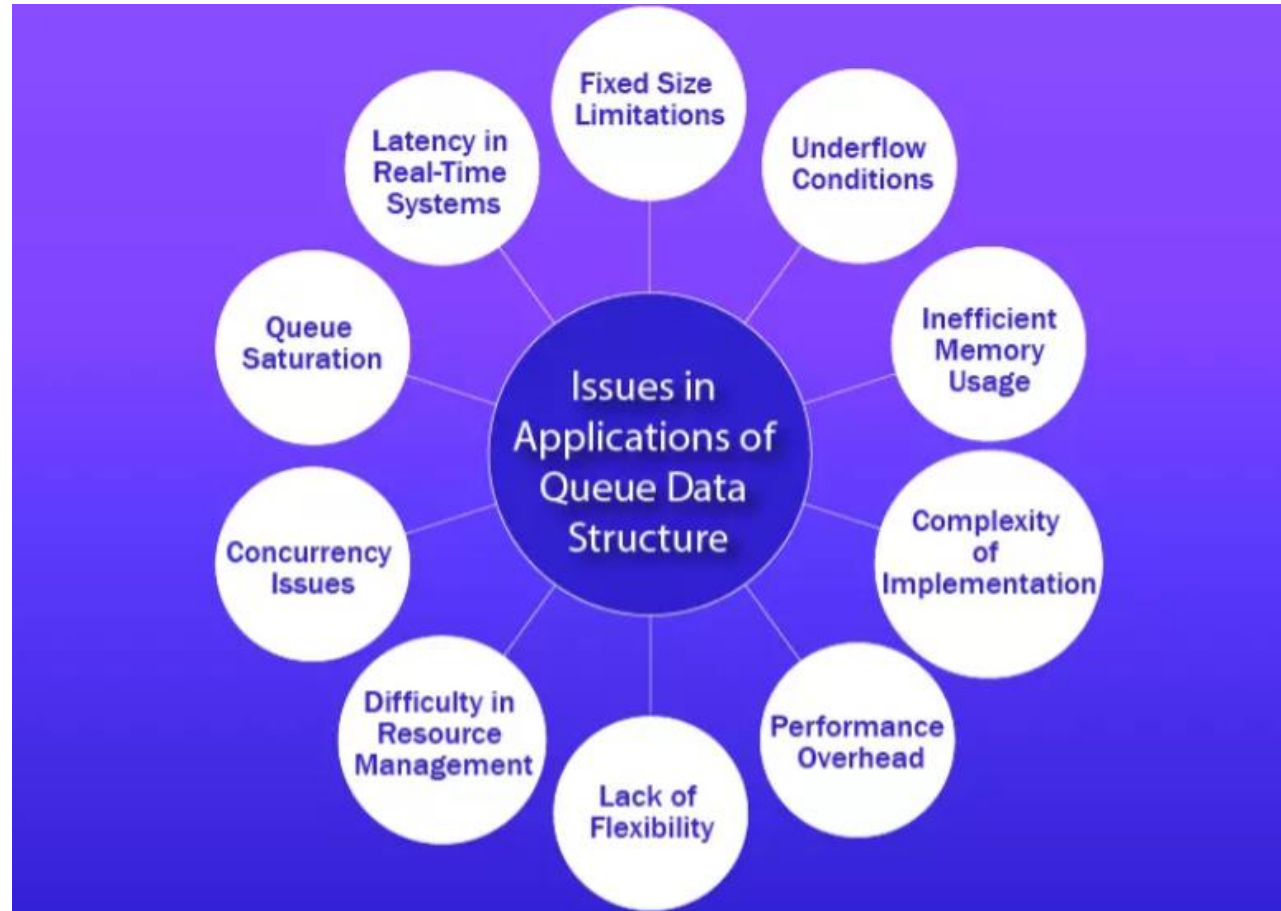
Queue:



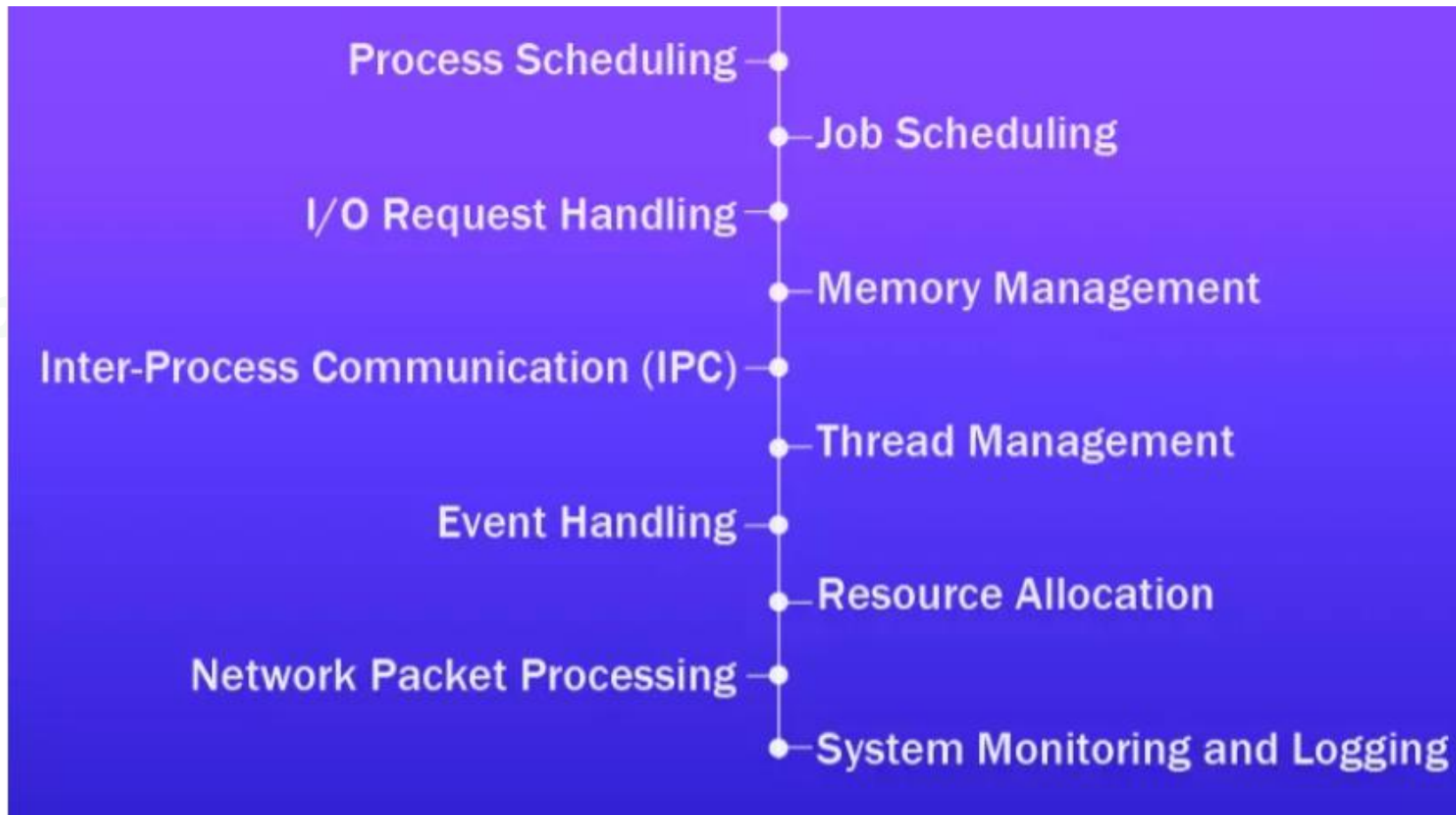
Operations:

Operation	Description	Time Complexity	Space Complexity
isEmpty	Returns true if the queue is empty, false otherwise.	$O(1)$	$O(1)$
Enqueue	This operation is used to add/insert data into the queue.	$O(1)$	$O(1)$
Dequeue	This operation is used to delete/remove data from the queue.	$O(1)$	$O(1)$
Peek	This operation returns the front element in the queue.	$O(1)$	$O(1)$

Issues in Applications of Queue:



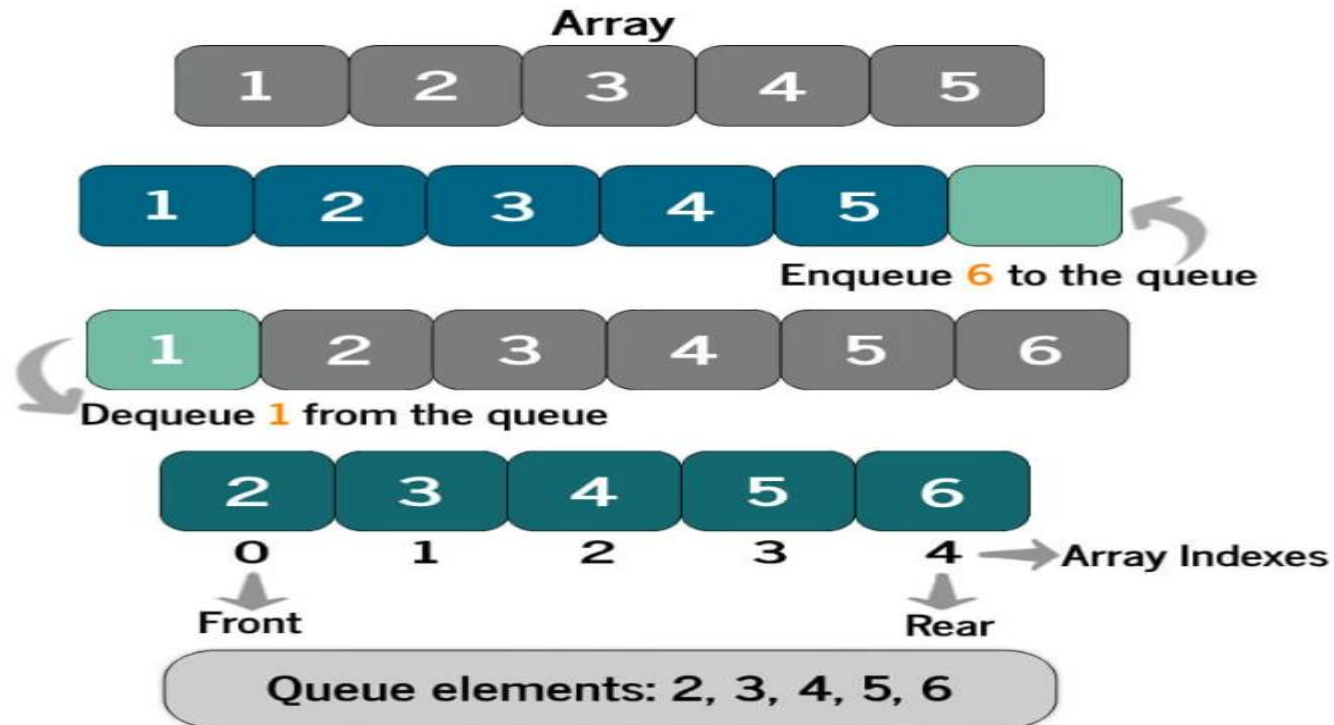
Applications:



Array Implementation of Queue:

- Array-Based Queue:
- A fixed-size array is used to store queue elements.
- Limitations:
 - Fixed size limits the number of elements.
 - If elements are dequeued from the front, the array does not reuse that space efficiently unless circular queues are used.

Array Implementation of Queue:

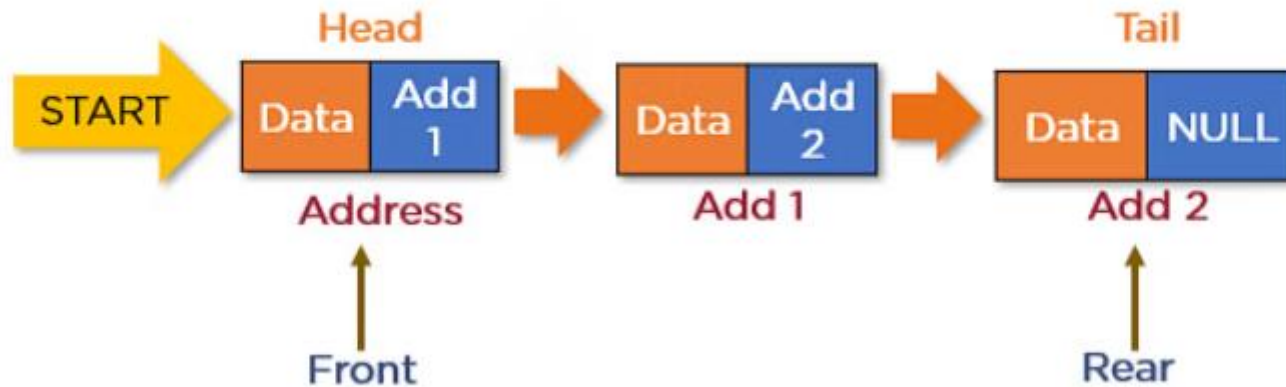


Linked List Implementation of Queue:

- Linked List-Based Queue:
- A dynamic data structure where elements are stored in nodes. Each node contains a data field and a reference to the next node.
- Advantages:
 - No fixed size; it can grow or shrink dynamically as elements are added or removed.
 - Efficient memory usage.

Linked List Implementation of Queue:

Linked List Representation of a Queue



Circular Queue (Array Implementation):

- Circular Queue Overview:
- Circular queue solves the inefficiency of the array-based queue by reusing the empty space once elements are dequeued.
- Key Features:
 - When the rear reaches the end of the array, it wraps around to the beginning if there is available space.

Priority Queue:

- A Priority Queue is a special type of queue where each element is assigned a priority, and the element with the highest priority is dequeued first.
- Types of Priority Queue:
 - Min Priority Queue: The element with the lowest value is given the highest priority.
 - Max Priority Queue: The element with the highest value is given the highest priority.
- Implementation:
 - Priority queues can be implemented using arrays, linked lists, or heaps (binary heaps are common for efficient operations).

Algorithm : PUSH(HEAD, DATA, PRIORITY):

- Step 1: Create new node with DATA and PRIORITY
- Step 2: Check if HEAD has lower priority. If true follow Steps 3-4 and end. Else goto Step 5.
- Step 3: NEW -> NEXT = HEAD
- Step 4: HEAD = NEW
- Step 5: Set TEMP to head of the list
- Step 6: While TEMP -> NEXT != NULL and TEMP -> NEXT -> PRIORITY > PRIORITY
- Step 7: TEMP = TEMP -> NEXT
- [END OF LOOP]
- Step 8: NEW -> NEXT = TEMP -> NEXT
- Step 9: TEMP -> NEXT = NEW
- Step 10: End

Algorithm :

- POP(HEAD):
 - Step 1: Set the head of the list to the next node in the list. $HEAD = HEAD \rightarrow NEXT$.
 - Step 2: Free the node at the head of the list
 - Step 3: End
- PEEK(HEAD):
 - Step 1: Return $HEAD \rightarrow DATA$
 - Step 2: End

Applications of Queue:

- Task Scheduling:
 - Operating systems use queues to manage tasks in multi-tasking environments.
- 2. Data Buffering:
 - Queues are used in buffers for IO devices (e.g., keyboards, printers).
- 3. Breadth-First Search (BFS):
 - Used in graph traversal algorithms like BFS.
- 4. Simulation Systems:
 - Queues are used in simulation models to handle customer service systems, traffic management, etc.

Advantages of Queue:

- A large amount of data can be managed efficiently with ease.
- Operations such as insertion and deletion can be performed with ease as it follows the first in first out rule.
- Queues are useful when a particular service is used by multiple consumers.
- Queues are fast in speed for data inter-process communication.
- Queues can be used in the implementation of other data structures.

Disadvantages of Queue:

- The operations such as insertion and deletion of elements from the middle are time consuming.
- In a classical queue, a new element can only be inserted when the existing elements are deleted from the queue.
- Searching an element takes $O(N)$ time.
- Maximum size of a queue must be defined prior in case of array implementation.